

Projekt z uczenia maszynowego

Nauczenie komputera sterowania samochodem w grze 2D

Sieć neuronowa jako kierowca pojazdu sterowanego
na podstawie sygnałów z czujników odległości

Raport końcowy

5 czerwca 2026

Spis treści

1	Cel projektu	2
2	Opis środowiska — gra w C++	2
2.1	Model fizyki pojazdu	2
2.2	Czujniki — „oczy” kierowcy	3
2.3	Zbieranie danych	4
3	Sformułowanie problemu uczenia maszynowego	4
3.1	Cechy wejściowe	4
3.2	Funkcja straty i nierównowaga klas	4
4	Dane i ich przygotowanie	5
4.1	Charakterystyka zbioru	5
4.2	Kroki przygotowania danych	5
4.3	Spójność uczenia i wdrożenia	6
5	Analiza eksploracyjna danych	6
5.1	Rozkład etykiet i nierównowaga klas	6
5.2	Wnioski z analizy zbalansowania	7
6	Architektura modelu	7
7	Metodyka eksperymentów	8
7.1	Podział danych i powtarzalność	8
7.2	Procedura treningu	8
7.3	Metryki oceny	8
7.4	Logika decyzyjna i progi	9
8	Model bazowy i strojenie konfiguracji	9
8.1	Model bazowy	9
8.2	Przeszukiwanie przestrzeni hiperparametrów	9
9	Wyniki	10
9.1	Przebieg uczenia modelu bazowego	10
9.2	Jakość poszczególnych akcji	11
9.3	Wyniki strojenia hiperparametrów	12
9.4	Porównanie z modelem bazowym	13
10	Wdrożenie modelu w grze	13
11	Analiza błędów i ograniczenia	14
12	Wnioski i kierunki dalszych prac	14

1 Cel projektu

Celem projektu jest **nauczenie komputera samodzielnego sterowania samochodem w grze 2D**. Zadaniem wytrenowanego modelu jest obserwowanie stanu pojazdu i jego otoczenia, a następnie podejmowanie w czasie rzeczywistym decyzji sterujących (przyspieszanie, hamowanie, skręt w lewo, skręt w prawo) tak, aby pojazd poruszał się po torze i dojechał do mety, nie uderzając w przeszkody.

Należy podkreślić, że *właściwym celem projektu nie jest samo testowanie wartości hiperparametrów*. Strojenie konfiguracji modelu (rozdział 8) jest jedynie **środkiem** prowadzącym do celu nadrzędnego — uzyskania modelu, który skutecznie zastępuje człowieka przy kierownicy. Eksperymenty z hiperparametrami traktujemy więc jako narzędzie poprawiające jakość kierowcy, a nie jako produkt sam w sobie.

Projekt obejmuje pełny proces uczenia maszynowego:

1. zebranie danych z gry (rozgrywka człowieka jako wzorzec),
2. przygotowanie i normalizację danych,
3. analizę eksploracyjną zebranego zbioru,
4. zaprojektowanie i wytrenowanie sieci neuronowej,
5. wyznaczenie modelu bazowego i jego optymalizację,
6. ocenę modelu metrykami klasyfikacji,
7. wdrożenie modelu w grze i sterowanie pojazdem w czasie rzeczywistym.

2 Opis środowiska — gra w C++

Środowiskiem projektu jest autorska gra napisana w języku C++ z wykorzystaniem biblioteki graficznej **raylib**. Gra przedstawia jazdę samochodem w widoku z góry (2D). Gracz buduje tor z klocków (linie barierowe, filary, zakręty oraz bloki typu start/meta/checkpoint), a następnie steruje pojazdem.

2.1 Model fizyki pojazdu

Pojazd opisany jest położeniem, kątem obrotu oraz prędkością skalarną. Sterowanie odbywa się czterema niezależnymi sygnałami (*Controls*): **accelerate**, **brake**, **steerLeft**, **steerRight**. Uproszczona dynamika (z pliku `Car.cpp`) zakłada m.in. naturalne tłumienie prędkości, ograniczenie prędkości do przedziału $[-5, 10]$ oraz zależność siły skrętu od prędkości:

```
1 void Car::updateSpeedRot(Controls inputs) {  
2     this->speed *= 0.995f; // naturalne  
   tłumienie  
3     if (inputs.accelerate && (this->speed < 10)) this->speed  
       += 0.1f;
```

```

4      if (inputs.brake      && (this->speed > -5))      this->speed
      -= 0.3f;
5      if (this->speed < -5) this->speed = -5;
6      if (this->speed > 10) this->speed = 10;
7
8      float maxTurn = 3.0f, turnSpeed = 0;
9      if (abs(this->speed) < 0.5f)      turnSpeed = 0;
10     else if (abs(this->speed) <= 5.0f) turnSpeed =
      (this->speed/5.0f)*maxTurn;
11     else turnSpeed = maxTurn*(1.0f -
      0.5f*((abs(this->speed)-5.0f)/5.0f));
12
13     if (inputs.steerLeft)  this->rotation -= turnSpeed;
14     if (inputs.steerRight) this->rotation += turnSpeed;
15 }

```

Listing 1: Aktualizacja prędkości i obrotu pojazdu (fragment `Car.cpp`).

Kolizja z barierą (typ 0) powoduje reset pojazdu do punktu odrodzenia, blok checkpoint (typ 2) aktualizuje punkt odrodzenia, a wjazd na metę (typ 3) zatrzymuje licznik czasu i kończy przejazd.

2.2 Czujniki — „oczy” kierowcy

Postrzeganie otoczenia przez pojazd zrealizowano za pomocą **20 promieni (raycastów)** rozchodzących się od środka samochodu. Trzydzieści promieni pokrywa przedni wachlarz (co 10°), a kolejnych siedem obserwuje tył i boki. Dla każdego promienia wyznaczana jest:

- **odległość** do najbliższego trafionego obiektu (`ray_dist`),
- **typ** trafionego obiektu (`ray_type`): -1 — brak trafienia, 0 — ściana/przeszkoda, 1 — meta.

Te 20 odległości oraz 20 typów wraz z prędkością tworzą pełny *stan gry* (`GameState`), który stanowi wejście modelu. Zestaw promieni jest aktualizowany względem aktualnego obrotu pojazdu:

```

1 void Car::UpdateRays() {
2     for (int i = 0; i < 13; i++)
3         this->Rays[i] = { cos((-60 + 10*i +
4                             this->rotation)*DEG2RAD),
5                             sin((-60 + 10*i +
6                                 this->rotation)*DEG2RAD) };
7     for (int i = 0; i < 7; i++)
8         this->Rays[i+13] = { cos((100 + 30*i +
9                                 this->rotation)*DEG2RAD),
10                             sin((100 + 30*i +
11                                 this->rotation)*DEG2RAD) };
12 }

```

Listing 2: Wyznaczanie kierunków promieni (fragment `Car.cpp`).

2.3 Zbieranie danych

Podczas jazdy sterowanej przez człowieka gra zapisuje do pliku CSV kolejne stany gry razem z wciśniętymi w danej klatce klawiszami. Nagłówek pliku generowany jest w `GameState.cpp` i ma postać:

```
car_speed, ray_dist_0, ray_type_0, ..., ray_dist_19, ray_type_19,  
  
accelerate, brake, steer_left, steer_right.
```

W ten sposób człowiek pełni rolę „nauczyciela”: model uczy się odwzorowywać decyzje gracza w zależności od sytuacji na torze (uczenie nadzorowane, ang. *imitation learning*).

3 Sformułowanie problemu uczenia maszynowego

Zadanie sterowania pojazdem sprowadzono do **klasyfikacji wieloetykietowej** (ang. *multi-label classification*). Dla danego stanu $\mathbf{x}$ model przewiduje cztery niezależne prawdopodobieństwa odpowiadające czterem klawiszom sterującym:

$$\hat{\mathbf{y}} = (p_{\text{acc}}, p_{\text{brake}}, p_{\text{left}}, p_{\text{right}}), \quad p_k \in (0, 1).$$

Etykiety nie są wzajemnie wykluczające się (np. można jednocześnie przyspieszać i skręcać), dlatego nie jest to klasyczna klasyfikacja jednoklasowa, lecz cztery sprzężone problemy binarne uczone wspólną siecią.

3.1 Cechy wejściowe

Wejście modelu budowane jest identycznie po stronie Pythona (uczenie) i C++ (wnioskowanie w grze). Składa się z **81 cech**:

- 1 cecha numeryczna: `car_speed`,
- 20 cech numerycznych: `ray_dist_0..19`,
- 60 cech binarnych: zakodowanie „one-hot” typu każdego z 20 promieni (3 możliwe wartości: $-1, 0, 1$).

Cechy numeryczne (`car_speed` oraz odległości) są standaryzowane (rozdział 4), natomiast cechy typu promienia, jako zmienne kategoryczne, kodowane są binarnie bez skalowania.

3.2 Funkcja straty i nierównowaga klas

Jako funkcję straty zastosowano `BCEWithLogitsLoss` (binarna entropia krzyżowa liczona bezpośrednio z logitów). Ponieważ klawisze sterujące występują

w danych z bardzo różną częstością (np. hamowanie jest rzadkie, a przyspieszanie niemal stałe), do każdej etykiety przypisano wagę pozytywnej klasy w_k (`pos_weight`):

$$w_k = \text{clip}\left(\frac{N_{\text{neg}}^{(k)}}{\max(N_{\text{pos}}^{(k)}, 1)}, 0,5, c\right),$$

gdzie c to górne ograniczenie wagi (hiperparametr `pos_weight_cap`). Mechanizm ten zapobiega ignorowaniu rzadkich, ale istotnych akcji (np. hamowania).

4 Dane i ich przygotowanie

4.1 Charakterystyka zbioru

Surowy zbiór (`GameStatesTable.csv`) zawiera ponad 65 000 wierszy zarejestrowanych w trakcie rozgrywki. Każdy wiersz to jedna klatka gry: stan czujników oraz odpowiadające mu decyzje gracza.

4.2 Kroki przygotowania danych

Skrypt przygotowujący dane (zaimplementowany w funkcjach `load_data` oraz `build_features`) wykonuje następujące operacje:

1. wczytanie pliku CSV i oczyszczenie nazw kolumn,
2. konwersję kolumn na typ numeryczny i usunięcie wierszy z brakami (NaN), co chroni m.in. przed przypadkowym powtórzeniem nagłówka,
3. binaryzację etykiet do wartości $\{0, 1\}$ progiem 0,5,
4. opcjonalne usunięcie duplikatów oraz początkowych „bezczynnych” klatek (gdy pojazd stoi i nie ma żadnego wejścia),
5. standaryzację cech numerycznych za pomocą `StandardScaler` (dopasowanego wyłącznie na zbiorze treningowym),
6. kodowanie one-hot typów promieni,
7. złączenie cech w macierz $\mathbf{X}$ o 81 kolumnach oraz wektor etykiet $\mathbf{y}$ o 4 kolumnach.

```
1 def build_features(df, scaler=None, fit_scaler=False):
2     numeric = df[NUMERIC_COLS].astype(np.float32).to_numpy()
3     if fit_scaler:
4         scaler = StandardScaler()
5         numeric =
6             scaler.fit_transform(numeric).astype(np.float32)
7     else:
8         numeric = scaler.transform(numeric).astype(np.float32)
9
10    type_features = []
11    for col in TYPE_COLS:
```

```

11     values = df[col].astype(int).to_numpy()
12     for ray_type in RAY_TYPE_VALUES: #
13         kolejnosc: -1, 0, 1
14         one_hot = (values ==
15                     ray_type).astype(np.float32).reshape(-1, 1)
16         type_features.append(one_hot)
17
18     x = np.concatenate([numeric] + type_features,
19                        axis=1).astype(np.float32)
20     return x, scaler

```

Listing 3: Budowa cech wejściowych: standaryzacja liczb i one-hot typów (fragment `train_driver_model.py`).

4.3 Spójność uczenia i wdrożenia

Aby model wytrenowany w Pythonie działał identycznie w grze C++, parametry standaryzacji (średnia i odchylenie) są eksportowane do nagłówka C++ (`AIScalerData.h`). Dzięki temu funkcja `BuildAIInputFromGameState` buduje wektor wejściowy w dokładnie tej samej kolejności i skali co skrypt uczący:

```

1 static float ScaleNumericInput(float value, int index) {
2     return (value - AI_INPUT_MEAN[index]) /
3           AI_INPUT_SCALE[index];
4 }
5
6 // 1x car_speed + 20x ray_dist + 20x one-hot(ray_type) => 81
7     cech

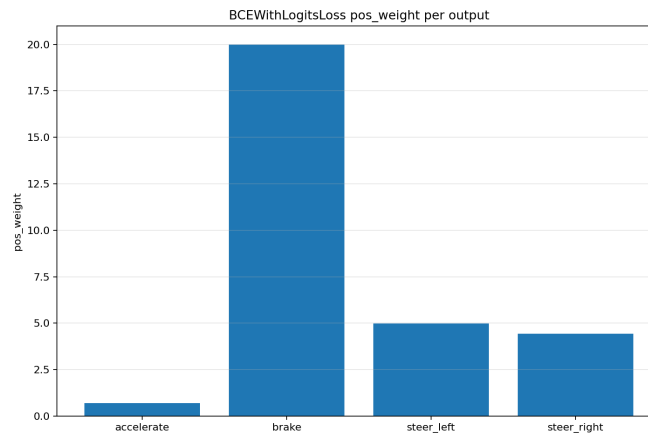
```

Listing 4: Standaryzacja cech po stronie gry (fragment `AIInputBuilder.cpp`).

5 Analiza eksploracyjna danych

5.1 Rozkład etykiet i nierównowaga klas

Analiza częstości wciśnieć klawiszy ujawnia silną **nierównowagę klas**. Przyspieszanie jest akcją dominującą, natomiast hamowanie pojawia się sporadycznie. Najlepiej obrazuje to wyznaczony wektor `pos_weight` (rysunek 1): waga dla hamowania osiąga górny limit (20), co oznacza, że bez korekty sieć ignorowałaby tę akcję. Skrety mają wagi rzędu 4–5, a przyspieszanie wagę poniżej 1 (klasa większościowa).



Rysunek 1: Wagi klasy pozytywnej (`pos_weight`) dla poszczególnych akcji. Bardzo wysoka waga hamowania odzwierciedla jego rzadkość w danych.

5.2 Wnioski z analizy zbalansowania

Z powodu nierównowagi klas **sama dokładność (accuracy) jest metryką mylącą**: model przewidujący zawsze „brak hamowania” osiągnąłby wysoką trafność dla tej etykiety, będąc bezużytecznym. Dlatego w ocenie modelu (rozdział 9) posługujemy się dodatkowo precyzją, czułością oraz miarą F1 liczonymi osobno dla każdej akcji, ze szczególnym uwzględnieniem hamowania. Z tego samego powodu w treningu zastosowano ważenie klas oraz strojenie progów decyzyjnych.

6 Architektura modelu

Wybrany model jest **wielowarstwowy perceptron (MLP)** zaimplementowany w bibliotece PyTorch (klasa `DriverNet`). Sieć przyjmuje 81 cech i zwraca 4 logity. W wariantcie podstawowym wykorzystano normalizację wsadową (*batch normalization*), funkcje aktywacji ReLU oraz warstwę dropout przeciwdziałającą przeuczeniu.

```

1 class DriverNet(nn.Module):
2     def __init__(self, input_size: int, dropout: float):
3         super().__init__()
4         self.net = nn.Sequential(
5             nn.Linear(input_size, 128),
6             nn.BatchNorm1d(128),
7             nn.ReLU(),
8             nn.Dropout(dropout),
9             nn.Linear(128, 128),
10            nn.ReLU(),
11            nn.Dropout(dropout),
12            nn.Linear(128, 64),

```

```

13         nn.ReLU(),
14         nn.Linear(64, 4),      # 4 logity: acc, brake,
                                left, right
15     )
16     def forward(self, x):
17         return self.net(x)

```

Listing 5: Architektura sieci DriverNet z konfigurowalnym dropoutem (fragment `grid_search_driver_basic.py`).

Sieć zwraca logity (bez sigmoidy), ponieważ funkcja straty `BCEWithLogitsLoss` łączy w sobie sigmoidę i entropię krzyżową w sposób numerycznie stabilny. Sigmoida nakładana jest dopiero na etapie wnioskowania.

7 Metodyka eksperymentów

7.1 Podział danych i powtarzalność

Zbiór dzielony jest na część treningową i walidacyjną (domyślnie 80/20). Gdy w danych dostępna jest kolumna identyfikująca przejazd (`episode_id`), podział wykonywany jest *grupowo* (całe przejazdy trafiają do jednego zbioru), co zapobiega wyciekowi informacji między klatkami tego samego przejazdu i daje uczciwszą ocenę. Powtarzalność zapewnia ustalone ziarno losowości (`set_seed`) dla bibliotek `random`, `numpy` i `torch`.

7.2 Procedura treningu

Trening wykorzystuje optymalizator `AdamW` (z regularyzacją *weight decay*) oraz mechanizm **wczesnego zatrzymania** (*early stopping*): zapamiętywany jest stan modelu o najmniejszej stracie walidacyjnej, a uczenie przerywane jest, gdy strata nie poprawia się przez ustaloną liczbę epok (*patience*).

7.3 Metryki oceny

Ze względu na nierównowagę klas oraz wieloetykietowy charakter problemu raportowane są:

- **strata walidacyjna** (`val_loss`),
- **dokładność dokładna** (*exact accuracy*) — odsetek klatek, w których wszystkie 4 akcje przewidziano poprawnie,
- **dokładność per-przycisk** dla każdej akcji,
- **precyzja, czułość, F1** dla każdej akcji oraz uśrednione (macro-F1),
- **PR-AUC** (pole pod krzywą precyzja–czułość) dla rzadkich klas.

7.4 Logika decyzyjna i progi

Surowe prawdopodobieństwa zamieniane są na konkretne sterowanie z uwzględnieniem zasad gry: **hamowanie ma priorytet nad gazem**, a skręt to wybór *lewo albo prawo*, a nie obie naraz. Progi decyzyjne są osobnym, ważnym elementem rozwiązania — są strojone na zbiorze walidacyjnym pod kątem F1, co często poprawia jakość sterowania bardziej niż dalsze dostrajanie wag sieci.

```
1 def action_predictions(y_prob, thresholds):
2     pred = np.zeros_like(y_prob, dtype=np.int64)
3     t_acc, t_brake, t_left, t_right = thresholds.tolist()
4     brake_mask = y_prob[:, 1] >= t_brake
5     acc_mask = (~brake_mask) & (y_prob[:, 0] >= t_acc)    #
6     pred[brake_mask, 1] = 1
7     pred[acc_mask, 0] = 1
8     left_active = y_prob[:, 2] >= t_left
9     right_active = y_prob[:, 3] >= t_right
10    steer_active = left_active | right_active
11    pred[steer_active & (y_prob[:, 2] >= y_prob[:, 3]), 2] = 1
12    # lewo ALBO prawo
13    pred[steer_active & (y_prob[:, 3] > y_prob[:, 2]), 3] = 1
14    return pred
```

Listing 6: Zamiana prawdopodobieństw na sterowanie z priorytetem hamowania (fragment `grid_search_driver_basic.py`).

8 Model bazowy i strojenie konfiguracji

8.1 Model bazowy

Punktem odniesienia jest model wytrenowany w skrypcie `train_driver_model.py` z ustalonymi parametrami (m.in. `lr=0,001`, `weight_decay=10-4`, `batch_size=512`, `dropout 0,1`, `pos_weight_cap=20`, do 80 epok). Stanowi on bazę, względem której oceniamy zyski z optymalizacji.

8.2 Przeszukiwanie przestrzeni hiperparametrów

Globalną optymalizację zrealizowano metodą *grid search* (`grid_search_driver_basic.py`). Przeszukiwana siatka obejmuje parametry o największym wpływie na jakość kierowcy:

Tabela 1: Przestrzeń przeszukiwania hiperparametrów.

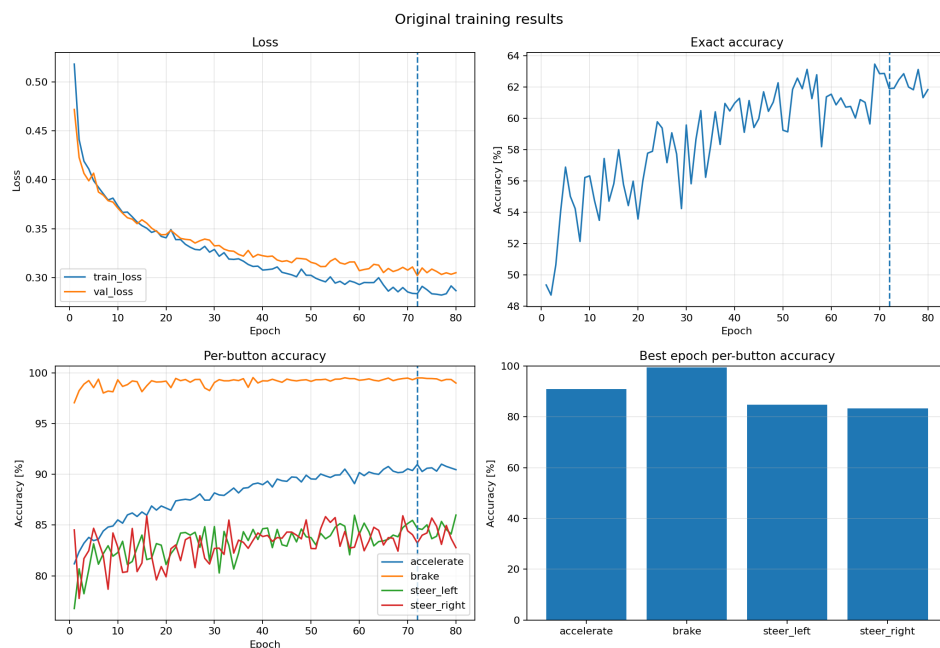
Hiperparametr	Testowane wartości
Współczynnik uczenia (<code>lr</code>)	10^{-3} , $5 \cdot 10^{-4}$
Regularyzacja (<code>weight_decay</code>)	10^{-4} , 10^{-3}
Dropout	0,05, 0,10, 0,20
Rozmiar wsadu (<code>batch_size</code>)	128, 256
Limit wagi klasy (<code>pos_weight_cap</code>)	5, 10, 20

Dla każdej konfiguracji trenowany jest osobny model, a następnie *dodatkowo* strojone są progi decyzyjne (rozdział 7.4). Wyniki zapisywane są do pliku CSV oraz rankingów Markdown, co zapewnia powtarzalność i przejrzystość analizy. **Przypominamy jednak, że strojenie to jedynie etap pośredni:** ostatecznym kryterium jest jakość jazdy najlepszego znalezionego kierowcy.

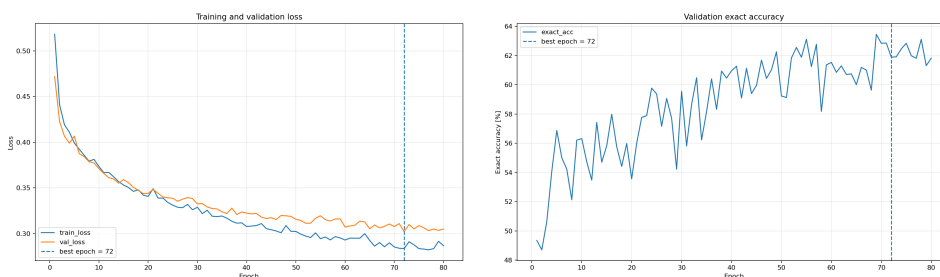
9 Wyniki

9.1 Przebieg uczenia modelu bazowego

Rysunek 2 zbiorczo przedstawia przebieg uczenia: spadek straty, wzrost dokładności dokładnej oraz dokładności poszczególnych akcji. Najlepszy stan modelu uzyskano w okolicy epoki 72.



Rysunek 2: Zbiorcze podsumowanie treningu modelu bazowego: strata, dokładność dokładna oraz dokładność per-przycisk wraz z wartościami w najlepszej epoce.



(a) Strata treningowa i walidacyjna.

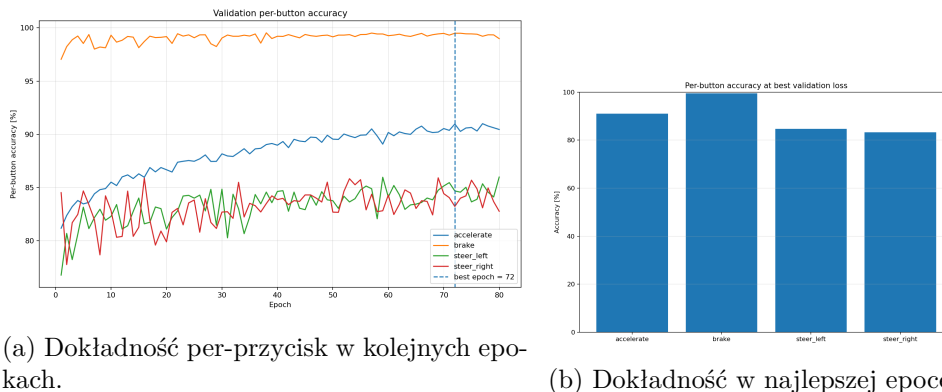
(b) Dokładność dokładna na walidacji.

Rysunek 3: Krzywe uczenia. Strata walidacyjna stabilizuje się ok. 0,30, a dokładność dokładna rośnie do ok. 62–63%. Niewielka różnica między stratą treningową a walidacyjną świadczy o umiarkowanym przeuczeniu.

9.2 Jakość poszczególnych akcji

Dokładność per-przycisk (rysunki 4a i 4b) pokazuje, że model najlepiej radzi sobie z hamowaniem (ok. 99% — tu jednak decyduje rzadkość klasy) oraz przyspieszaniem (ok. 91%). Trudniejsze okazują się skrety (ok. 83–85%), co jest zgodne z intuicją: decyzja o skręcie wymaga subtelniejszej interpretacji

układu promieni niż decyzja o jeździe na wprost.



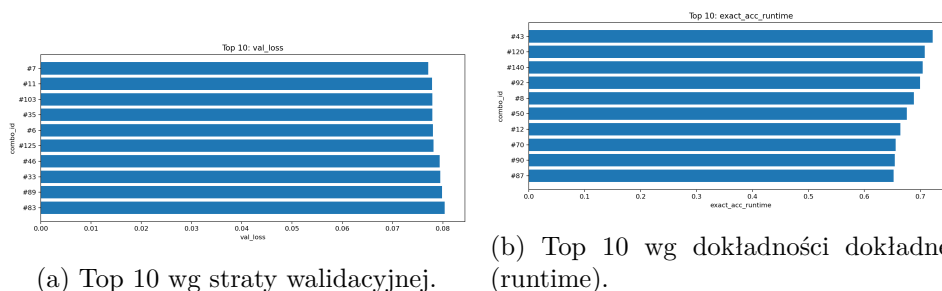
(a) Dokładność per-przycisk w kolejnych epokach.

(b) Dokładność w najlepszej epoce.

Rysunek 4: Dokładność dla poszczególnych akcji. Skręt w lewo i w prawo to akcje najtrudniejsze do nauczenia.

9.3 Wyniki strojenia hiperparametrów

Ranking konfiguracji według straty walidacyjnej (rysunek 5a) pokazuje, że wiele konfiguracji daje bardzo zbliżone wyniki (val_loss ok. 0,077–0,080), co świadczy o stabilności architektury względem testowanych hiperparametrów. Ranking według dokładności dokładnej w trybie runtime (rysunek 5b) wskazuje najlepsze konfiguracje przekraczające 70% poprawnie odtworzonych pełnych zestawów akcji.

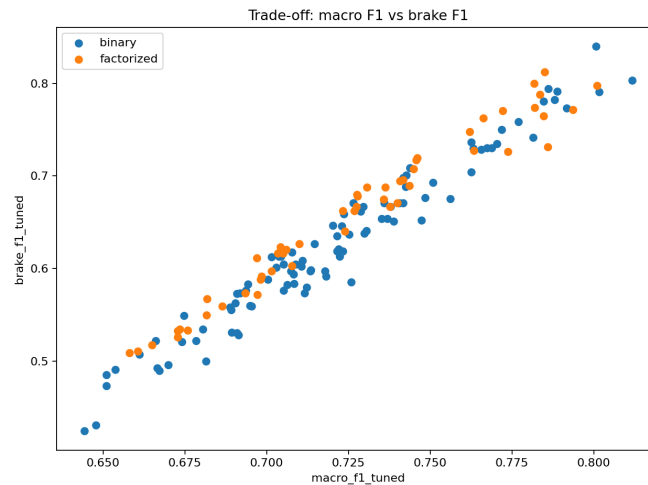


(a) Top 10 wg straty walidacyjnej.

(b) Top 10 wg dokładności dokładnej (runtime).

Rysunek 5: Rankingi konfiguracji z przeszukiwania siatki hiperparametrów.

Rysunek 6 ilustruje zależność między uśrednioną miarą F1 (macro-F1) a miarą F1 dla samego hamowania. Widać silną dodatnią korelację — konfiguracje dobre „ogólnie” są zwykle dobre także w trudnej akcji hamowania, choć dla najlepszych modeli pojawia się pewien kompromis między tymi metrykami.



Rysunek 6: Kompromis między macro-F1 a F1 dla hamowania (po strojeniu progów). Każdy punkt to jedna konfiguracja modelu.

9.4 Porównanie z modelem bazowym

Najlepsza konfiguracja wyłoniona w przeszukiwaniu osiąga niższą stratę walidacyjną i wyższe macro-F1 niż model bazowy, przy zachowaniu wysokiej skuteczności hamowania. Zysk wynika w dużej mierze z **łącnego** dostrojenia wag sieci oraz progów decyzyjnych, a nie z pojedynczego hiperparametru — co potwierdza, że celem jest sprawny kierowca, a nie optymalizacja jednej liczby.

10 Wdrożenie modelu w grze

Wytrenowany model eksportowany jest do formatu **ONNX** i ładowany w grze przez klasę `NeuralDriver` (biblioteka ONNX Runtime). W każdej klatce gra:

1. buduje 81-elementowy wektor wejściowy ze stanu pojazdu (`BuildAIInputFromGameState`),
2. uruchamia model i otrzymuje 4 logity,
3. nakłada sigmoidę, uzyskując prawdopodobieństwa,
4. stosuje progi i zasady decyzyjne (priorytet hamowania, wybór jednego kierunku skrętu), zamieniając prawdopodobieństwa na sterowanie.

```

1 Controls NeuralDriver::PredictControls(const GameState& state)
  {
2     std::vector<float> probs = PredictProbabilities(state);
3     float pAccelerate = probs[0], pBrake = probs[1];
4     float pSteerLeft = probs[2], pSteerRight = probs[3];
5

```

```

6     Controls controls{};
7     if (pBrake > 0.50f)           controls.brake = 1.0f;
8         // priorytet hamulca
9     else if (pAccelerate > 0.30f) controls.accelerate = 1.0f;
10
11     float steeringThreshold = 0.35f;
12     if (pSteerLeft > steeringThreshold || pSteerRight >
13         steeringThreshold) {
14         if (pSteerLeft > pSteerRight) controls.steerLeft =
15             1.0f; // lewo ALBO prawo
16         else controls.steerRight =
17             1.0f;
18     }
19     return controls;
20 }

```

Listing 7: Zamiana prawdopodobieństw na sterowanie w grze (fragment NeuralDriver.cpp).

Przełączanie między kierowcą-człowiekiem a kierowcą-AI odbywa się w grze klawiszem M. Dzięki spójnemu budowaniu wejścia i identycznym progom decyzyjnym po obu stronach (Python i C++) zachowanie modelu w grze odpowiada wynikom z walidacji.

11 Analiza błędów i ograniczenia

- **Najczęściej mylone akcje:** skręt w lewo i skręt w prawo. Model bywa niezdecydowany w sytuacjach symetrycznych, gdy oba kierunki mają zbliżone prawdopodobieństwo — stąd reguła wyboru jednego (silniejszego) kierunku.
- **Imitacja, nie optymalizacja jazdy:** model uczy się odwzorowywać decyzje gracza, więc dziedziczy jego błędy i nie eksploruje lepszych strategii niewystępujących w danych.
- **Zależność od jakości danych:** sytuacje rzadkie na torze (ostre zakręty, cofanie) są słabo reprezentowane, co obniża skuteczność modelu w takich momentach.
- **Wyciek informacji:** przy losowym podziale na poziomie pojedynczych klatek kolejne, niemal identyczne klatki tego samego przejazdu mogą trafić do treningu i walidacji jednocześnie. Podział grupowy po przejazdach ogranicza ten efekt.

12 Wnioski i kierunki dalszych prac

Wnioski.

- Udało się **nauczyć komputer sterowania samochodem** w grze 2D — sieć neuronowa na podstawie 20 promieni i prędkości podejmuje sensowne decyzje sterujące w czasie rzeczywistym.
- Kluczowe okazały się: ważenie rzadkich klas (**pos_weight**) oraz strojenie progów decyzyjnych, a nie samo dostrajanie wag sieci.
- Najtrudniejszym elementem jazdy są skrety; hamowanie, mimo rzadkości, jest dobrze rozpoznawane dzięki ważeniu klas.
- Przeszukiwanie hiperparametrów potraktowano jako narzędzie poprawy kierowcy — różnice między najlepszymi konfiguracjami są niewielkie, co świadczy o stabilności rozwiązania.

Dalsze prace.

- zastosowanie **uczenia ze wzmocnieniem** (RL), aby model optymalizował czas przejazdu, a nie jedynie naśladował gracza,
- dodanie informacji o **historii** (kilka poprzednich klatek lub sieć rekurencyjna) dla płynniejszego sterowania,
- zbieranie danych z wielu torów i wielu graczy w celu lepszej generalizacji,
- rejestrowanie identyfikatora przejazdu (**episode_id**) dla rzetelnego podziału grupowego i uczciwszej oceny.

Literatura

- [1] A. Paszke i in., *PyTorch: An Imperative Style, High-Performance Deep Learning Library*, NeurIPS, 2019.
- [2] F. Pedregosa i in., *Scikit-learn: Machine Learning in Python*, JMLR, 2011.
- [3] ONNX Runtime, *Open Neural Network Exchange Runtime*, <https://onnxruntime.ai>.
- [4] R. Santamaria, *raylib — a simple and easy-to-use library to enjoy videogames programming*, <https://www.raylib.com>.